

## EXPERIMENT NO. 5

### Build a real-time search filter for a list of items using JavaScript strings/arrays

Name: Rohit Pujari      Roll No.: 28      Date: \_\_\_\_\_

#### Aim:

To build “BookShelf”, a college library catalogue of 20 books that is filtered on every keystroke by title and author, where a query of several words must match all the words in any order; genre chips generated from the data narrow the list, the result can be sorted by title, year or rating, the matched words are highlighted, and the current search is kept in the page URL so that it can be bookmarked or shared.

#### Theory:

The catalogue is a JavaScript array of objects, each with title, author, genre, year and rating. The search box listens to the input event, which fires after every character that is typed, pasted or deleted. On each event the program builds a new array with a chain of array methods and redraws the list. The original array is never modified: `slice()` makes a copy before `sort()`, because `sort()` changes the array it is called on.

#### **Multi-word matching with `split()` and `every()`**

The query is converted to lower case and split on white space with `split(/\s+/)`; `filter(Boolean)` removes the empty strings produced by extra spaces. For each book the title and author are joined into one lower-case string, and `every(word => text.includes(word))` is true only if all the words occur in it. Therefore “rowling harry” finds both Harry Potter books although the words are in a different order from the title. For an empty query the word list is empty, and `every()` on an empty array returns true, so all books are shown without any special case.

| Method                                                   | Purpose in BookShelf          | Example                                        |
|----------------------------------------------------------|-------------------------------|------------------------------------------------|
| <code>split(/\s+/)</code>                                | Break the query into words    | "rowling harry" -> ["rowling", "harry"]        |
| <code>filter(Boolean)</code>                             | Drop empty strings            | ["", "code"] -> ["code"]                       |
| <code>every(fn)</code>                                   | All words must be present     | true for 2 of the 20 books                     |
| <code>includes(s)</code>                                 | Substring test                | "clean code robert c. martin".includes("code") |
| <code>[...new Set(arr)]</code>                           | Unique genres for the chips   | 20 books -> 7 genres                           |
| <code>reduce(fn, 0)</code>                               | Average rating of the results | $(4.5 + 4.4) / 2 = 4.45$                       |
| <code>localeCompare()</code>                             | Alphabetical sort             | "Ikigai" before "Sapiens"                      |
| <code>replace(re, "&lt;mark&gt;\$1&lt;/mark&gt;")</code> | Highlight matched words       | Harry -> <mark>Harry</mark>                    |

#### **Genre chips and sorting**

The chips are not written in the HTML; they are generated from the data with `new Set()`, so adding a book of a new genre automatically adds a chip, and the count on each chip is found with `filter().length`. Instead of one listener per chip, a single click listener on the chip container reads `event.target.dataset.genre` (event delegation). The sort order comes from an object of comparator functions: title uses `localeCompare()`, year uses `b.year - a.year` for newest first, and rating sorts by rating and breaks ties by title.

#### **Safe highlighting with regular expressions**

The matched words are wrapped in `<mark>` using one regular expression built from all query words, with the flags `g` (all matches) and `i` (ignore case); the capture group `$1` keeps the original capitalisation. Because the words come from the user, characters such as `+` `( )` `.` `*` that have a special meaning in a RegExp are first escaped with a backslash, so a query such as “c++ (x)” cannot throw an error. The book text is HTML-escaped before the `<mark>` tags are inserted.

#### **Keeping the search in the URL**

- **URLSearchParams** builds a query string such as `?q=code&genre=Programming&sort=rating` from the current state and reads it back when the page loads.
- **history.replaceState()** updates the address bar without reloading the page and without adding one history entry per keystroke, so the Back button still leaves the page.
- **Keyboard shortcuts:** “/” focuses the search box and Esc clears it.

## Testing the filter

| Test                           | Expected result                              | Observed               |
|--------------------------------|----------------------------------------------|------------------------|
| Type "rowling harry"           | 2 of 20 books, words highlighted             | As expected (Fig. 5.1) |
| Open ?q=code&genre=Programming | State restored from URL, 1 book (Clean Code) | As expected            |
| Type "xyz"                     | Message "No books match ..."                 | As expected            |
| Type "c++ (x"                  | No error, 0 results                          | As expected            |
| Press Esc, choose All          | All 20 books shown                           | As expected            |

## Important Code:

### JavaScript - escaping and highlighting

```
const escapeRegex = s => s.replace(/[*+?^$(){}|\[\]\\\]/g, '\\$&');

function highlight(text, words) {
  const safe = escapeHtml(text);
  if (!words.length) return safe;
  const re = new RegExp('(' + words.map(w => escapeRegex(escapeHtml(w))).join('|') + ')',
    'gi');
  return safe.replace(re, '<mark>$1</mark>');
}
```

### JavaScript - filtering, sorting and the URL

```
function render() {
  const words = state.q.toLowerCase().split(/\s+/).filter(Boolean); // "rowling harry"
  -> 2 words
  const list = books
    .filter(b => state.genre === 'All' || b.genre === state.genre)
    .filter(b => words.every(w => (b.title + ' ' + b.author).toLowerCase().includes(w)))
    .slice().sort(sorters[state.sort]);
  // ... list and chips are rendered here ...
  const p = new URLSearchParams();
  if (state.q) p.set('q', state.q);
  if (state.genre !== 'All') p.set('genre', state.genre);
  if (state.sort !== 'title') p.set('sort', state.sort);
  history.replaceState(null, '', p.toString() ? '?' + p : location.pathname); // shareable URL
}
```

### JavaScript - events

```
$('#q').addEventListener('input', e => { state.q = e.target.value; render(); });
$('#sort').addEventListener('change', e => { state.sort = e.target.value; render(); });
$('#chips').addEventListener('click', e => {
  if (e.target.dataset.genre) { state.genre = e.target.dataset.genre; render(); }
});
```

## Output:

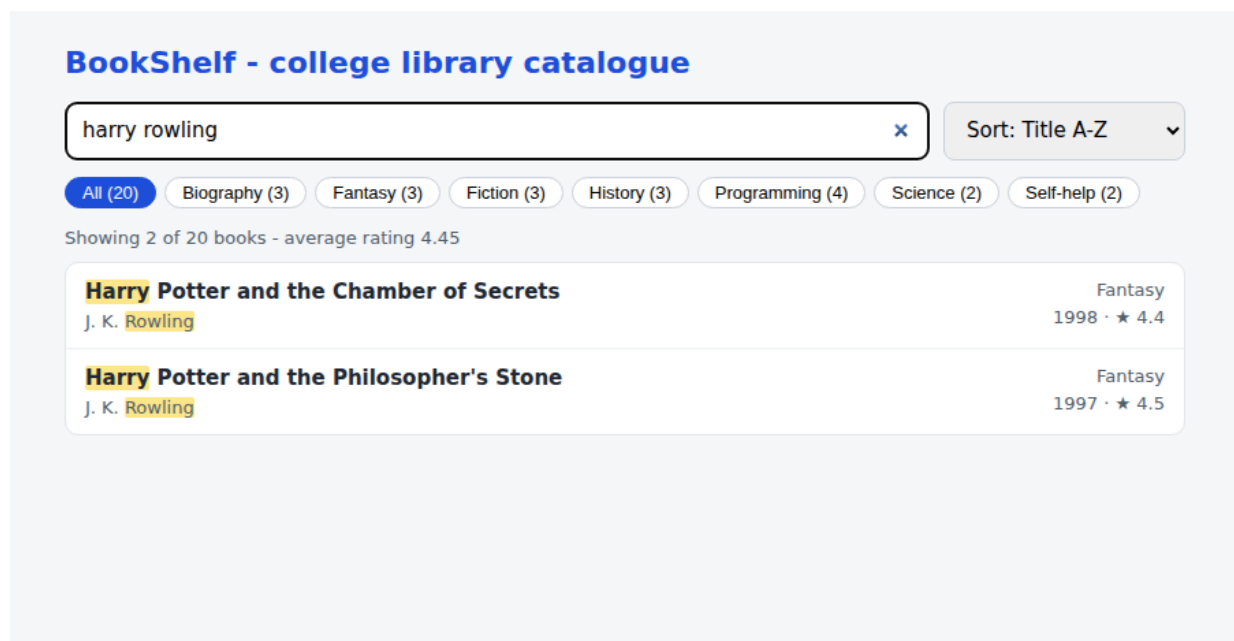


Figure 5.1: Query “harry rowling” - 2 of 20 books match both words, matched words highlighted

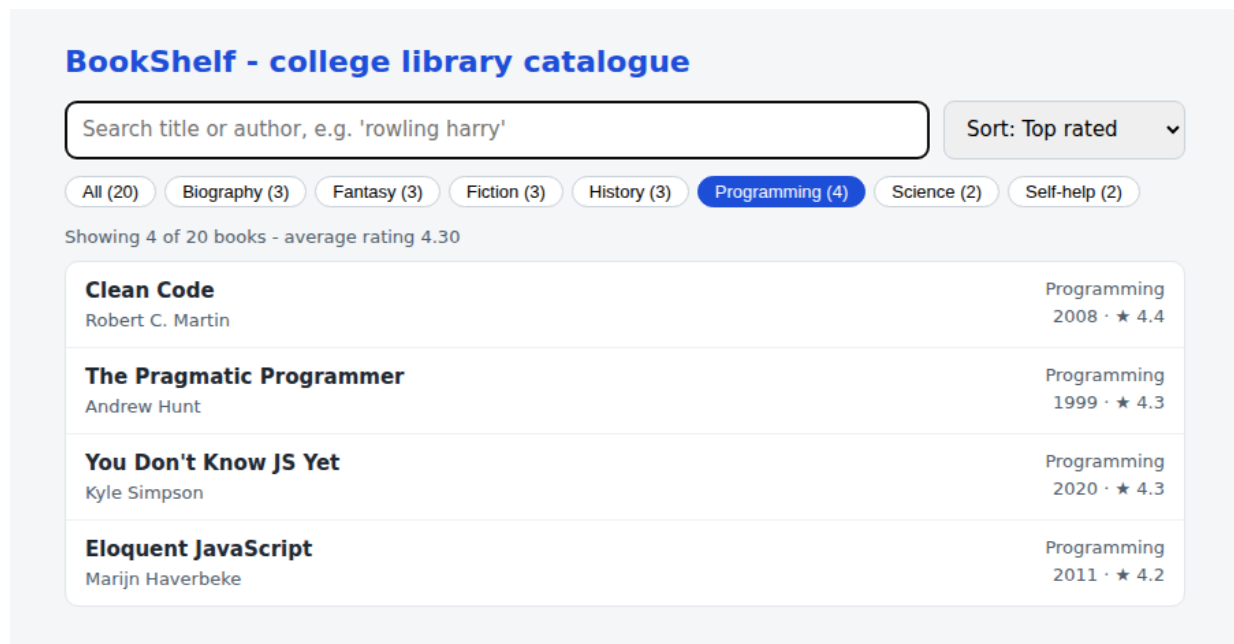


Figure 5.2: Genre chip “Programming” with the sort order “Top rated”

## Conclusion:

A real-time book search was built using only JavaScript string and array methods. `split()` and `every()` allowed multi-word queries in any order, `filter()`, `sort()` and `reduce()` produced the result list and its average rating, a Set generated the genre chips, an escaped regular expression highlighted matches safely, and `URLSearchParams` with `history.replaceState()` kept the search in the URL.